A PG LEVEL PROJECT REPORT

ON

PEER PROJECT TEAM FORMATION & SKILL-GAP MATCHER

Submitted by

Sevakula Pavithra – 26MAI0049

Duggu Sujana – 26MAI0056

Under the Guidance of

Prof. Dr. Ramanathan L

Professor Grade 1



VELLORE INSTITUTE OF TECHNOLOGY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

VELLORE, TAMILNADU

September 2026

TABLE OF CONTENTS

ABSTRACT i
LIST OF FIGURES ii
LIST OF TABLES ii
1. INTRODUCTION 1
1.1 Project Overview 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Organization of Report 2
1.6 Feasibility Study 2
2. LITERATURE SURVEY 4
2.1 Existing Systems 4
2.2 Limitations of Existing Systems 4
2.3 Proposed System 4
3. SYSTEM REQUIREMENTS 5
3.1 Functional Requirements 5
3.2 Non-Functional Requirements 6
3.3 Hardware & Software Requirements 6
4. SYSTEM DESIGN 8
4.1 ER Diagram 8
4.2 Database Schema Design 10
4.3 Normalization - up to 3NF 15
4.4 Data Dictionary 16
5. IMPLEMENTATION 18
5.1 Database Implementation - DDL, DML 18
5.2 Views, Triggers, Stored Procedures 19

ABSTRACT

The Peer Project Team Formation & Skill-Gap Matcher is a web-based platform that helps students assemble project teams by matching project requirements against student skills, interests, experience, preferred roles and availability, rather than relying on convenience-based, informal team formation which commonly produces skill duplication and coverage gaps.

Students maintain a profile describing their skills and proficiency levels, browse available projects, and apply for specific roles. Project owners define required skills, roles and team size, and review incoming applications. At the core of the system is a complementarity-based matching algorithm that scores each candidate by the marginal improvement they would bring to a team's current skill coverage, so that a candidate who fills a genuine gap is ranked above one who merely repeats a skill the team already has - a distinction that simple keyword-matching tools cannot make.

Team composition is made transparent through a skill-gap radar chart comparing required and covered skills, a Team Balance Index (0–100) summarising skill coverage, role diversity, experience mix and availability overlap, and a short natural-language explanation accompanying every recommendation.

The system is implemented on Firebase Cloud Firestore (NoSQL), with Firebase Authentication for identity and Firebase Cloud Functions automating the recomputation of skill coverage, the Team Balance Index and candidate recommendations whenever underlying data changes. The front end is built in React and the backend in Node.js/Express. The matching approach is evaluated on a synthetic dataset of 1,500+ records against random-assignment and keyword-matching baselines, and is shown to reduce residual skill gaps and improve team balance relative to both.

LIST OF FIGURES

Figure 1: Entity-Relationship Diagram 8
Figure 2: Logical Schema Diagram 9

LIST OF TABLES

Table 1: Functional Requirements
Table 2: Non-Functional Requirements
Table 3: Hardware Requirements
Table 4: Software & Technology Stack
Table 5: Entity–Relationship Summary
Table 6: USERS - Table Schema
Table 7: STUDENT_PROFILES - Table Schema
Table 8: SKILLS - Table Schema
Table 9: STUDENT_SKILLS - Table Schema
Table 10: ROLES - Table Schema
Table 11: STUDENT_PREFERRED_ROLES - Table Schema
Table 12: AVAILABILITY_SLOTS - Table Schema
Table 13: PROJECTS - Table Schema
Table 14: PROJECT_REQUIRED_SKILLS - Table Schema
Table 15: PROJECT_REQUIRED_ROLES - Table Schema
Table 16: TEAMS - Table Schema
Table 17: TEAM_MEMBERS - Table Schema
Table 18: APPLICATIONS - Table Schema
Table 19: RECOMMENDATIONS - Table Schema
Table 20: SKILL_GAP_SNAPSHOTS - Table Schema
Table 21: TEAM_BALANCE_HISTORY - Table Schema
Table 22: Firestore Collection Mapping (Data Dictionary)
Table 23: Synthetic Dataset - Record Volume
Table 24: Cloud Function Procedures Summary
Table 25: Firestore Composite Indexes

1. INTRODUCTION

1.1 Project Overview

Academic courses that involve group projects typically leave team formation to the students themselves. In practice, students gravitate toward friends, roommates or classmates they already know rather than teammates whose skills complement the project at hand. This convenience-based approach is fast, but it routinely produces teams with duplicated strengths and gaps in skills the project genuinely needs. The Peer Project Team Formation & Skill-Gap Matcher is a web-based platform, built in the Education Technology (EdTech) domain, that replaces this informal process with a structured, skill-aware matching system.

Students maintain a profile describing their technical skills, proficiency levels, interests, prior project experience, preferred roles and weekly availability. Project owners create projects and specify the skills, roles and team size required. The system's complementarity-based matching engine then ranks candidates by the marginal improvement they would bring to a team's current skill coverage, rather than by how many required skills a candidate personally holds - a distinction existing keyword-matching tools cannot make. The result is surfaced through a skillgap radar chart, an explainable recommendation list, and a single Team Balance Index summarising overall team health.

1.2 Problem Statement

There is no lightweight, skill-aware system available to student groups and course coordinators that (a) captures each student's technical skills, proficiency, interests, availability and preferred roles in one place, (b) captures each project's required skills, roles and team size, and (c) recommends teammates based on the marginal value they would add to a specific team's current skill coverage, rather than on raw keyword overlap or personal familiarity. As a result, teams formed by existing means are frequently unbalanced: some required skills are left completely uncovered, workload distribution is uneven, and neither students nor project owners have a clear, explainable view of why a particular teammate was, or was not, a good fit.

1.3 Objectives

- Enable students to create profiles highlighting their skills, proficiency, interests, project experience, preferred roles, and availability.
- Allow project owners to post projects, specify skill requirements, team size, and manage applications.
- Develop a complementarity-based matching algorithm that prioritizes candidates who address existing team skill gaps rather than duplicate strengths.
- Visualize project skill gaps using radar charts, including the potential impact of adding candidates.

- Calculate a composite Team Balance Index (0–100) based on skill coverage, role diversity, experience, and availability overlap.
- Provide clear, human-readable explanations for each recommendation to ensure transparency.
- Implement the platform using Firebase Cloud Firestore, Authentication, Cloud Functions, React, and Node.js/Express, and evaluate its performance on 1,500+ synthetic records against random-assignment and keyword-matching baselines.

1.4 Scope of the Project

The system is scoped to academic project team formation within a single institution or course context. It covers student and project-owner registration and profile management; project creation and discovery; skill and role taxonomies; the application and acceptance workflow; the complementarity-matching engine and candidate recommendation service; the skill-gap radar and Team Balance Index visualisations; and an analytics dashboard summarising team health across projects. It excludes payroll, academic grading, plagiarism detection of the project work produced by teams, and direct messaging between teammates, which are treated as out of scope for the current version.

Target Users

- Students - create a profile, browse open projects, apply for roles, and preview how well they fit a project before applying.
- Project Owners - students or faculty-appointed leads who create projects, define requirements, and review ranked, explainable candidate recommendations.
- Course Coordinators / Faculty (secondary) - view aggregate analytics on teamformation quality across a batch of projects.

1.5 Organization of Report

Chapter 2 reviews existing approaches to peer team formation and positions the proposed system relative to their limitations. Chapter 3 specifies the system's functional and non-functional requirements together with its hardware and software needs. Chapter 4 presents the system design: the entity-relationship model, the normalized logical schema, and the mapping of that schema onto the chosen Firestore data store. Chapter 5 describes the database implementation - how NoSQLnative mechanisms stand in for DDL, DML, views, triggers and stored procedures - covering the work completed through Section 5.2.

1.6 Feasibility Study

Technical Feasibility

All chosen technologies - React, Node.js/Express, Firebase Cloud Firestore, Firebase Authentication and Firebase Cloud Functions - are mature, well-documented, free-tier-accessible platforms with active community support, and fall within the skill set expected of the project team.

The complementarity-matching logic is a set-coverage/marginal-gain computation implementable in standard JavaScript without specialised infrastructure.

Economic Feasibility

The project uses only free tiers of its chosen platforms (Firebase Spark plan, GitHub, open-source npm packages), so it can be built and demonstrated without licensing cost; Firebase Hosting is free for low-traffic academic use.

Operational Feasibility

The required workflows - creating a profile, posting a project, applying, and reviewing recommendations - mirror processes students already know from job/internship portals, keeping the learning curve low. Firebase Cloud Functions automate the recomputation of derived values, so the system can be operated without dedicated database-administration effort.

2. LITERATURE SURVEY

2.1 Existing Systems

Three broad categories of tool are currently used, formally or informally, to form academic project teams:

- Manual / word-of-mouth formation - the default in most courses. Students self-select teammates from friends or classmates. It is fast and requires no tooling, but does not scale once cohort size or the number of concurrent projects grows.
- Generic group-finder or forum-style tools - allow students to post interest in a project and wait for others to respond, but provide no structured model of skills, proficiency or availability, and no ranking of candidates.
- Keyword-based matching tools - the closest existing analogue to the proposed system. A candidate is scored by how many of a project's required skills they personally list, similar in spirit to a resume keyword search.

2.2 Limitations of Existing Systems

- None of the above evaluate a candidate relative to what a specific team already has; a keyword-matching tool would rank a fourth Python developer as highly as a project's first, even though the team gains little from the duplication.
- None provide a visual comparison of required versus covered skills, so an owner cannot see at a glance which requirements remain unmet.
- None produce a single, composite indicator of team health (skill coverage, role diversity, experience mix, availability overlap combined), forcing an owner to reason about several disconnected signals manually.
- None explain why a particular candidate was suggested, which makes it difficult for an owner to trust or audit a recommendation, or for a rejected candidate to understand the decision.
- Manual formation does not scale with cohort size, and cross-checking skills against requirements by hand becomes impractical beyond a single classroom.

2.3 Proposed System

The Peer Project Team Formation & Skill-Gap Matcher addresses these gaps by treating team formation as a set-coverage problem rather than a per-candidate keyword-matching problem. Each candidate is scored by the marginal gain they would bring to the team's current skill coverage, combined with role fit, availability overlap and experience match. The result is surfaced through an explainable recommendation, a skill-gap radar chart, and a Team Balance Index, giving project owners both a ranked list of candidates and the reasoning behind each rank - directly addressing every limitation identified in Section 2.2.

3. SYSTEM REQUIREMENTS

3.1 Functional Requirements

Requirements are grouped by the workflow they support. Each is uniquely numbered so it can be traced to the schema (Chapter 4) and to the implementation (Chapter 5).

ID	Requirement
FR-1	The system shall allow a new user to register with a name, email and password via Firebase Authentication.
FR-2	The system shall allow a registered user to log in and log out.
FR-3	The system shall allow a student to create/edit a profile with department, year of study, experience level, bio, skills with proficiency levels, ranked preferred roles, and weekly availability.
FR-4	The system shall allow a user to hold both a student profile and a project-owner role simultaneously.
FR-5	The system shall allow a project owner to create a project with a title, description, domain area, required skills (with minimum proficiency), required roles, target team size and deadline.
FR-6	The system shall allow a project owner to edit or close a project.
FR-7	The system shall allow any student to browse and filter open projects by domain, required skill or role.
FR-8	The system shall allow a student to apply to an open project for a specific role.
FR-9	The system shall allow a project owner to view all applications for their project, sorted by match/marginal-gain score.
FR-10	The system shall allow a project owner to accept or reject an application; acceptance shall add the applicant to the project's team.
FR-11	The system shall notify a student in-app when their application status changes.
FR-12	The system shall compute, for each candidate, a marginal-gain score representing the improvement to team skill coverage if that candidate is added, rather than scoring the candidate's skills in isolation.
FR-13	The system shall combine marginal skill-gain with role fit, availability overlap and experience match into a single ranked recommendation score.
FR-14	The system shall generate a short natural-language explanation for each recommended candidate.
FR-15	The system shall allow a project owner to preview the effect on team skill coverage of adding a specific candidate before accepting their application.
FR-16	The system shall display a radar chart comparing a project's required skills against the team's currently covered skills.
FR-17	The system shall compute and display a Team Balance Index (0–100) combining skill coverage, role diversity, experience mix and availability overlap.

FR-18	The system shall automatically recompute skill coverage, the Team Balance Index and candidate recommendations whenever relevant profile, project or team-membership data changes.
FR-19	The system shall provide a dashboard summarising Team Balance Index trends and skill-gap status across a project owner's or coordinator's projects.
FR-20	The system shall support running the matching engine against a synthetic dataset of 500+ records and reporting results against a random-assignment baseline and a keyword-matching baseline.

Table 1: Functional Requirements

3.2 Non-Functional Requirements

Category	Requirement
Performance	Recommendation results for up to 100 candidate applicants shall return in under 2 seconds, using precomputed skill-coverage values maintained by Cloud Functions.
Usability	Core workflows shall be completable by a first-time user without external instructions, using a responsive React UI.
Reliability	Firestore read/write failures shall be handled with a user-facing retry/error message rather than a silent failure.
Security	Passwords and sessions shall be managed exclusively through Firebase Authentication; Firestore Security Rules shall restrict each user to their own authorised documents.
Scalability	The data model shall support at least 500 student and project records without requiring schema changes.
Maintainability	The matching engine shall be implemented as an isolated, unit-testable module separate from route/trigger handlers.
Portability	The front end shall run unmodified on the latest two major versions of Chrome, Edge and Firefox.
Availability	The application shall be reachable via Firebase Hosting with no scheduled downtime for normal operation.

Table 2: Non-Functional Requirements

3.3 Hardware & Software Requirements

Component	Minimum Requirement
Development machine	Dual-core CPU, 8 GB RAM, 10 GB free disk space
Client device	Any desktop or mobile device with an evergreen web browser and internet access
Server-side compute	Fully managed - Firebase Cloud Functions (serverless; no dedicated server to provision)

Network	Broadband internet connection (HTTPS access to Firebase services)

Table 3: Hardware Requirements

Layer	**Technology**
Frontend	React.js
Visualisation	Recharts / Chart.js
Backend	Node.js (Express)
Database (NoSQL)	Firebase Cloud Firestore
Authentication	Firebase Authentication
Database Automation	Firebase Cloud Functions
Version Control	Git + GitHub
Hosting	Firebase Hosting
Development Tools	Visual Studio Code, Postman, npm, Jest

Table 4: Software & Technology Stack

4. SYSTEM DESIGN

4.1 ER Diagram

The physical database chosen for implementation, Firebase Cloud Firestore, is schemaless and document-oriented. The conceptual data requirements are nonetheless first modelled as a normalized relational schema - up to Third Normal Form - to verify that no redundancy or update anomaly exists in the underlying data requirements before that schema is mapped onto Firestore collections (Section 4.4). The diagram below shows all sixteen entities identified for the system, their primary keys (PK, underlined), foreign keys (FK), and the cardinality of each relationship.

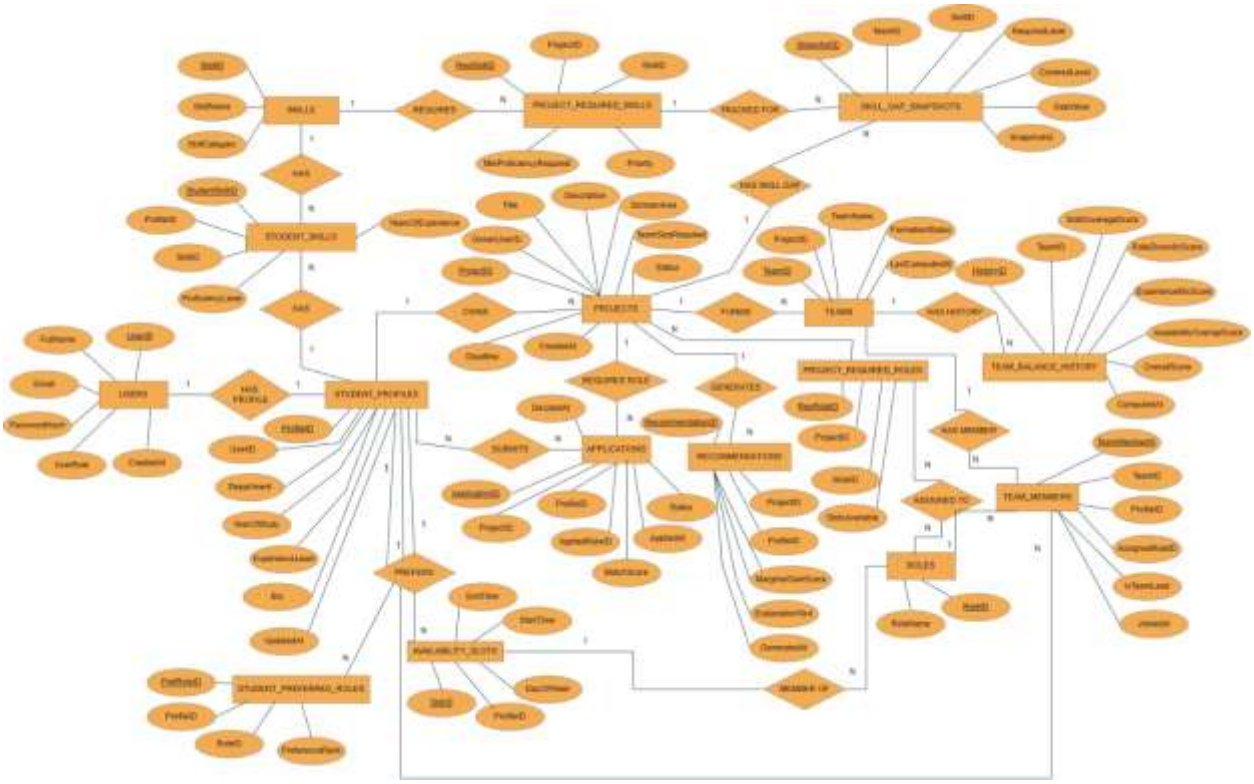


Figure 1: Entity-Relationship Diagram

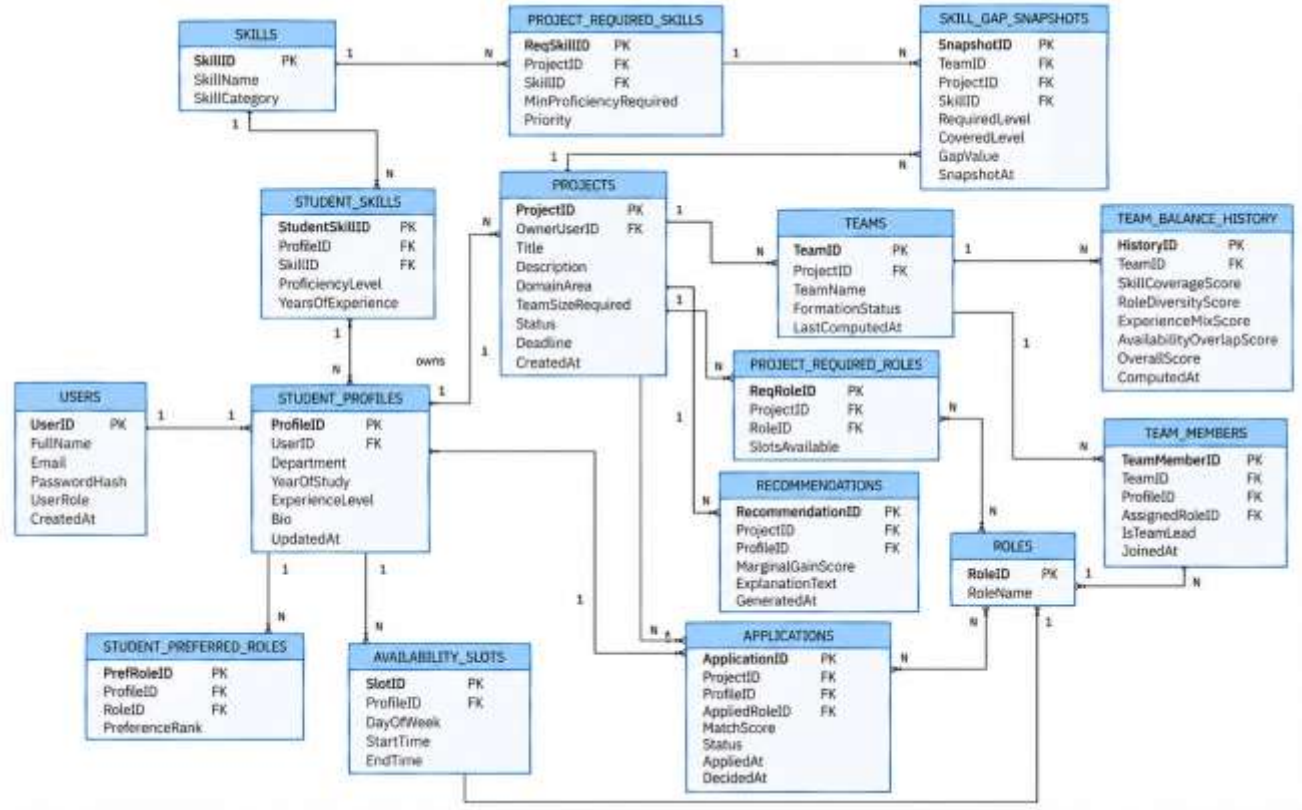


Figure 2: Logical Schema Diagram

	Relationship	**Cardinality**
1	USERS ↔ STUDENT_PROFILES	1 : 1
2	STUDENT_PROFILES ↔ SKILLS (via STUDENT_SKILLS)	M : N
3	STUDENT_PROFILES ↔ ROLES (via STUDENT_PREFERRED_ROLES)	M : N
4	STUDENT_PROFILES → AVAILABILITY_SLOTS	1 : N
5	USERS → PROJECTS	1 : N
6	PROJECTS ↔ SKILLS (via PROJECT_REQUIRED_SKILLS)	M : N
7	PROJECTS ↔ ROLES (via PROJECT_REQUIRED_ROLES)	M : N
8	PROJECTS ↔ TEAMS	1 : 1
9	TEAMS ↔ STUDENT_PROFILES (via TEAM_MEMBERS)	M : N
10	PROJECTS ↔ STUDENT_PROFILES (via APPLICATIONS)	M : N

11	PROJECTS ↔ STUDENT_PROFILES (via RECOMMENDATIONS)	M : N
12	TEAMS ↔ SKILLS (via SKILL_GAP_SNAPSHOTS)	M : N
13	TEAMS → TEAM_BALANCE_HISTORY	1 : N

Table 5: Entity–Relationship Summary

4.2 Database Schema Design

Data types are shown in generic SQL-like notation for clarity of constraints. Section 4.4 shows how each table is subsequently realised as a Firestore collection or sub-collection.

USERS

Attribute	Data Type	Key / Constraint	Notes
UserID	VARCHAR(36)	PK	UUID
FullName	VARCHAR(100)	NOT NULL	
Email	VARCHAR(150)	UNIQUE, NOT NULL	Managed by Firebase Auth
PasswordHash	VARCHAR(255)	NOT NULL	Managed by Firebase Auth
UserRole	ENUM	NOT NULL	Student / Owner / Both
CreatedAt	TIMESTAMP	NOT NULL	

Table 6: USERS - Table Schema

STUDENT_PROFILES

Attribute	Data Type	Key / Constraint	Notes
ProfileID	VARCHAR(36)	PK	UUID
UserID	VARCHAR(36)	FK → USERS.UserID, UNIQUE	1:1 with Users
Department	VARCHAR(80)	NOT NULL	
YearOfStudy	TINYINT	CHECK (1–5)	
ExperienceLevel	ENUM	NOT NULL	Beginner/Intermediate/ Advanced
Bio	TEXT	NULLABLE	
UpdatedAt	TIMESTAMP	NOT NULL	

Table 7: STUDENT_PROFILES - Table Schema

SKILLS

Attribute	Data Type	Key / Constraint	Notes
SkillID	VARCHAR(36)	PK	UUID
SkillName	VARCHAR(60)	UNIQUE, NOT NULL	e.g. React, Python
SkillCategory	VARCHAR(40)	NOT NULL	e.g. Frontend, Backend

Table 8: SKILLS - Table Schema

STUDENT_SKILLS

Attribute	Data Type	Key / Constraint	Notes
StudentSkillID	VARCHAR(36)	PK	UUID
ProfileID	VARCHAR(36)	FK → STUDENT_PROFILES.ProfileID	
SkillID	VARCHAR(36)	FK → SKILLS.SkillID	
ProficiencyLevel	TINYINT	CHECK (1–5), NOT NULL	
YearsOfExperience	DECIMAL(3,1)	NOT NULL	
		UNIQUE (ProfileID, SkillID)	Resolves M:N

Table 9: STUDENT_SKILLS - Table Schema

ROLES

Attribute	Data Type	Key / Constraint	Notes
RoleID	VARCHAR(36)	PK	UUID
RoleName	VARCHAR(60)	UNIQUE, NOT NULL	e.g. Frontend Dev, PM

Table 10: ROLES - Table Schema

STUDENT_PREFERRED_ROLES

Attribute	Data Type	Key / Constraint	Notes
PrefRoleID	VARCHAR(36)	PK	UUID
ProfileID	VARCHAR(36)	FK → STUDENT_PROFILES.ProfileID	
RoleID	VARCHAR(36)	FK → ROLES.RoleID	
PreferenceRank	TINYINT	NOT NULL	1 = most preferred
		UNIQUE (ProfileID, RoleID)	Resolves M:N

Table 11: STUDENT_PREFERRED_ROLES - Table Schema

AVAILABILITY_SLOTS

Attribute	Data Type	Key / Constraint	Notes
SlotID	VARCHAR(36)	PK	UUID
ProfileID	VARCHAR(36)	FK → STUDENT_PROFILES.ProfileID	
DayOfWeek	ENUM	NOT NULL	Mon–Sun
StartTime	TIME	NOT NULL	
EndTime	TIME	CHECK (EndTime > StartTime)	

Table 12: AVAILABILITY_SLOTS - Table Schema

PROJECTS

Attribute	Data Type	Key / Constraint	Notes
ProjectID	VARCHAR(36)	PK	UUID
OwnerUserID	VARCHAR(36)	FK → USERS.UserID	
Title	VARCHAR(120)	NOT NULL	
Description	TEXT	NOT NULL	
DomainArea	VARCHAR(60)	NOT NULL	
TeamSizeRequired	TINYINT	CHECK (> 0)	
Status	ENUM	NOT NULL	Open/InProgress/Closed
Deadline	DATE	NOT NULL	
CreatedAt	TIMESTAMP	NOT NULL	

Table 13: PROJECTS - Table Schema

PROJECT_REQUIRED_SKILLS

Attribute	Data Type	Key / Constraint	Notes
ReqSkillID	VARCHAR(36)	PK	UUID
ProjectID	VARCHAR(36)	FK → PROJECTS.ProjectID	
SkillID	VARCHAR(36)	FK → SKILLS.SkillID	
MinProficiencyRequired	TINYINT	CHECK (1–5)	
Priority	ENUM	NOT NULL	Must-have/Nice-to-have
		UNIQUE (ProjectID, SkillID)	Resolves M:N

Table 14: PROJECT_REQUIRED_SKILLS - Table Schema

PROJECT_REQUIRED_ROLES

Attribute	Data Type	Key / Constraint	Notes
ReqRoleID	VARCHAR(36)	PK	UUID
ProjectID	VARCHAR(36)	FK → PROJECTS.ProjectID	
RoleID	VARCHAR(36)	FK → ROLES.RoleID	
SlotsAvailable	TINYINT	CHECK (> 0)	
		UNIQUE (ProjectID, RoleID)	Resolves M:N

Table 15: PROJECT_REQUIRED_ROLES - Table Schema

TEAMS

Attribute	Data Type	Key / Constraint	Notes
TeamID	VARCHAR(36)	PK	UUID
ProjectID	VARCHAR(36)	FK → PROJECTS.ProjectID, UNIQUE	1:1 with Project
TeamName	VARCHAR(100)	NOT NULL	
FormationStatus	ENUM	NOT NULL	Forming/Complete
LastComputedAt	TIMESTAMP	NULLABLE	Set by Cloud Function

Table 16: TEAMS - Table Schema

TEAM_MEMBERS

Attribute	Data Type	Key / Constraint	Notes
TeamMemberID	VARCHAR(36)	PK	UUID
TeamID	VARCHAR(36)	FK → TEAMS.TeamID	
ProfileID	VARCHAR(36)	FK → STUDENT_PROFILES.ProfileID	
AssignedRoleID	VARCHAR(36)	FK → ROLES.RoleID	
IsTeamLead	BOOLEAN	NOT NULL, DEFAULT FALSE	
JoinedAt	TIMESTAMP	NOT NULL	
		UNIQUE (TeamID, ProfileID)	Resolves M:N

Table 17: TEAM_MEMBERS - Table Schema

APPLICATIONS

Attribute	Data Type	Key / Constraint	Notes
ApplicationID	VARCHAR(36)	PK	UUID

ProjectID	VARCHAR(36)	FK → PROJECTS.ProjectID	
ProfileID	VARCHAR(36)	FK → STUDENT_PROFILES.ProfileID	
AppliedRoleID	VARCHAR(36)	FK → ROLES.RoleID	
MatchScore	DECIMAL(5,2)	NOT NULL	System-computed
Status	ENUM	NOT NULL	Pending/Accepted/ Rejected/Withdrawn
AppliedAt	TIMESTAMP	NOT NULL	
DecidedAt	TIMESTAMP	NULLABLE	

Table 18: APPLICATIONS - Table Schema

RECOMMENDATIONS

Attribute	**Data Type**	**Key / Constraint**	**Notes**
RecommendationID	VARCHAR(36)	PK	UUID
ProjectID	VARCHAR(36)	FK → PROJECTS.ProjectID	
ProfileID	VARCHAR(36)	FK → STUDENT_PROFILES.ProfileID	
MarginalGainScore	DECIMAL(5,2)	NOT NULL	
ExplanationText	VARCHAR(255)	NOT NULL	Generated, human-readable
GeneratedAt	TIMESTAMP	NOT NULL	

Table 19: RECOMMENDATIONS - Table Schema

SKILL_GAP_SNAPSHOTS

Attribute	**Data Type**	**Key / Constraint**	**Notes**
SnapshotID	VARCHAR(36)	PK	UUID
TeamID	VARCHAR(36)	FK → TEAMS.TeamID	
SkillID	VARCHAR(36)	FK → SKILLS.SkillID	
RequiredLevel	TINYINT	NOT NULL	
CoveredLevel	TINYINT	NOT NULL	
GapValue	TINYINT	NOT NULL	= max(0, Required−Covered)

SnapshotAt	TIMESTAMP	NOT NULL	Set by Cloud Function

Table 20: SKILL_GAP_SNAPSHOTS - Table Schema

TEAM_BALANCE_HISTORY

Attribute	Data Type	Key / Constraint	Notes
HistoryID	VARCHAR(36)	PK	UUID
TeamID	VARCHAR(36)	FK → TEAMS.TeamID	
SkillCoverageScore	DECIMAL(5,2)	NOT NULL	0–100
RoleDiversityScore	DECIMAL(5,2)	NOT NULL	0–100
ExperienceMixScore	DECIMAL(5,2)	NOT NULL	0–100
AvailabilityOverlapScore	DECIMAL(5,2)	NOT NULL	0–100
OverallScore	DECIMAL(5,2)	NOT NULL	Weighted composite (TBI)
ComputedAt	TIMESTAMP	NOT NULL	Set by Cloud Function

Table 21: TEAM_BALANCE_HISTORY - Table Schema

4.3 Normalization - up to 3NF

First Normal Form (1NF)

Every attribute across all sixteen entities holds a single, atomic value. Multi-valued facts are never stored as delimited strings in one field; a student's skills, for instance, are not stored as a commaseparated list inside STUDENT_PROFILES. Instead, each (student, skill) fact is its own row in STUDENT_SKILLS, and each (student, availability-slot) fact its own row in AVAILABILITY_SLOTS, removing repeating groups.

Second Normal Form (2NF)

2NF requires every non-key attribute to depend on the whole of a composite key, not part of it. The junction tables - STUDENT_SKILLS, STUDENT_PREFERRED_ROLES, PROJECT_REQUIRED_SKILLS, PROJECT_REQUIRED_ROLES and TEAM_MEMBERS - are each given a single-column surrogate primary key with a UNIQUE constraint over the natural composite (e.g. ProfileID + SkillID). An attribute such as ProficiencyLevel depends on the combination of student and skill together, never on either alone, so no attribute depends on a subset of its table's key.

Third Normal Form (3NF)

3NF requires that no non-key attribute depend transitively on the primary key through another nonkey attribute. SkillName and SkillCategory are therefore stored once, in SKILLS, and referenced by SkillID everywhere else, rather than repeated inside STUDENT_SKILLS or PROJECT_REQUIRED_SKILLS - repeating them there would make SkillName transitively dependent on the key via SkillID and risk an update anomaly if a skill were renamed. The same reasoning keeps RoleName confined to ROLES, and Department/ExperienceLevel confined to STUDENT_PROFILES rather than duplicated per skill or application. With these dependencies removed, the schema is in 3NF.

4.4 Data Dictionary

The table below is the authoritative Firestore realisation of the 3NF schema in Section 4.2 - it is referenced, not repeated, throughout Chapter 5. Firestore has no joins, so a deliberate, limited degree of denormalization is applied on top of the normalized model: small, bounded facts are embedded directly in a parent document (e.g. a project's required skills), while facts that grow without bound become sub-collections (e.g. a project's applications). Cloud Functions (Section 5.2) keep any duplicated read-optimisation fields consistent with their source of truth.

Collection Path	Purpose	Key Fields
roles/{roleId}	Master list of team roles	roleName
skills/{skillId}	Master list of skills	skillName, skillCategory
users/{userId}	One document per account; student profile embedded as a map	fullName, email, userRole, profile: {department, yearOfStudy, experienceLevel}
users/{userId}/skills/{skillId}	Sub-collection: this student's skills	proficiencyLevel, yearsOfExperience, skillName
users/{userId}/ preferredRoles/{roleId}	Sub-collection: ranked role preferences	preferenceRank, roleName
users/{userId}/availability/ {slotId}	Sub-collection: weekly availability	dayOfWeek, startTime, endTime
projects/{projectId}	One document per project; required skills/roles embedded as arrays; team summary embedded as a map	title, ownerUserId, status, requiredSkills[], requiredRoles[], team:{teamSize, teamBalanceIndex}
projects/{projectId}/ applications/{applicationId}	Sub-collection: applications received	profileId, appliedRoleId, matchScore, status

projects/{projectId}/ teamMembers/{profileId}	Sub-collection: accepted team roster	assignedRoleId, isTeamLead, joinedAt
projects/{projectId}/ recommendations/{profileId}	Sub-collection: system-generated candidate rankings	marginalGainScore, explanationText
projects/{projectId}/team/ skillGap	Single materialized document: radar-chart data	skills:{skillId:{requiredLevel, coveredLevel, gapValue}}
projects/{projectId}/ tbiHistory/{historyId}	Sub-collection: append-only Team Balance Index log	skillCoverageScore, roleDiversityScore, experienceMixScore, availabilityOverlapScore, overallScore
analytics/skillDemand	Materialized view: skill demand across open projects	demand:{skillName: count}
analytics/ teamBalanceOverview	Materialized view: average TBI across all teams	averageTBI, teamsCounted

Table 22: Firestore Collection Mapping (Data Dictionary)

Referential integrity, which a relational engine would enforce with FOREIGN KEY constraints, is enforced here through Firestore Security Rules validating that a referenced document exists and belongs to the requesting user, and through the Cloud Functions of Section 5.2 that keep denormalized copies and derived aggregates consistent on every write.